implement C function which will take x, y coordinates of a starting point on Mercator projection map, heading in degrees, distance in meters and a scale factor and then return x, y coordinates of a destination point

Yes — the clean way is: convert the Mercator x, y start point back to latitude/longitude, move by the given heading and distance on the sphere, then project the destination latitude/longitude back to Mercator x, y. The Mercator forward/inverse formulas and destination-point formulas are standard and fit this use case well.[1] [2]

C function

The function below assumes a spherical Mercator projection with Earth radius 6378137 m, which is the usual Web Mercator radius, and treats `scale` as a multiplier applied to projected meters when returning map coordinates. Mercator uses $x = R\lambda$ and $y = R\ln(\tan(\pi/4 + \phi/2))$, while the destination point from start latitude/longitude, distance, and bearing uses the standard great-circle formulas with $\delta = d/R$.[2] [1]

```
#include <math.h>

#ifndef M_PI
#define M_PI 3.14159265358979323846
#endif

typedef struct {
    double x;
    double y;
} PointXY;

static double deg2rad(double deg) {
    return deg * M_PI / 180.0;
}

static double normalize_lon(double lon_rad) {
    while (lon_rad > M_PI)  lon_rad -= 2.0 * M_PI;
    while (lon_rad < -M_PI) lon_rad += 2.0 * M_PI;
    return lon_rad;
}

/*
 * start_x, start_y  - Mercator projected coordinates of start point
 * heading_deg       - bearing clockwise from north
```

```
 * distance_m        - travel distance in meters
 * scale             - map scale factor applied to projected coordinates
 *
 * Assumptions:
 * - Spherical Mercator / Web Mercator
 * - Input x,y are in "scaled Mercator meters":
 *       x = scale * R * lon
 *       y = scale * R * ln(tan(pi/4 + lat/2))
 *
 * Returns destination x,y in the same scaled coordinate system.
 */
PointXY mercator_destination(double start_x,
                             double start_y,
                             double heading_deg,
                             double distance_m,
                             double scale)
{
    const double R = 6378137.0;
    PointXY out = {0.0, 0.0};

    if (scale == 0.0) {
        return out;
    }

    /* Unscale projected coordinates back to Mercator meters */
    double mx = start_x / scale;
    double my = start_y / scale;

    /* Inverse Mercator: x,y -> lon,lat */
    double lon1 = mx / R;
    double lat1 = 2.0 * atan(exp(my / R)) - M_PI / 2.0;

    /* Great-circle destination formula */
    double brng = deg2rad(heading_deg);
    double delta = distance_m / R;

    double sin_lat1 = sin(lat1);
    double cos_lat1 = cos(lat1);
    double sin_delta = sin(delta);
    double cos_delta = cos(delta);
    double sin_brng = sin(brng);
    double cos_brng = cos(brng);

    double lat2 = asin(sin_lat1 * cos_delta +
                       cos_lat1 * sin_delta * cos_brng);

    double lon2 = lon1 + atan2(sin_brng * sin_delta * cos_lat1,
                               cos_delta - sin_lat1 * sin(lat2));

    lon2 = normalize_lon(lon2);

    /* Forward Mercator: lon,lat -> x,y */
    double dest_mx = R * lon2;
    double dest_my = R * log(tan(M_PI / 4.0 + lat2 / 2.0));

    /* Re-apply scale */
```

```
    out.x = dest_mx * scale;
    out.y = dest_my * scale;

    return out;
}
```

How `scale` is used

If your map coordinates are stored as projected meters directly, pass `scale = 1.0`. If your map uses pixels or some other linear unit derived from Mercator meters, `scale` should be the multiplier between Mercator meters and your coordinate system, because Mercator's projected coordinates are linear in x and y.[1]

For example, if your map uses `screen_x = mercator_x * 0.25` and `screen_y = mercator_y * 0.25`, then call the function with `scale = 0.25`, and the returned point will be in the same screen-space coordinate system. This works because the function removes the scale before geographic math and reapplies it after projecting the destination.[1]

Important note

If you actually want a constant-heading path on the Mercator map, that is a rhumb line, not a great-circle path. On Mercator, rhumb lines are straight lines and use a different destination formula than the shortest-path great-circle formulas used above.[2] [1]

If you want, I can also give you:

- a rhumb-line version for Mercator navigation,
- a WGS84 ellipsoidal version with better accuracy,
- or a version that takes/returns latitude and longitude too.

⁂

1. https://www.neacsu.net/docs/geodesy/snyder/numerics/mercator/
2. https://www.movable-type.co.uk/scripts/latlong.html
3. https://en.wikipedia.org/wiki/Mercator_projection
4. https://gist.github.com/perrygeo/4478844
5. https://neacsu.net/geodesy/snyder/numerics/mercator/
6. https://www.ancientportsantiques.com/wp-content/uploads/Documents/ETUDESarchivees/MedNavigationRoutes/MedNav/TrigoSpherique.pdf
7. https://gist.github.com/fgcarto/a24d89067f15551c622c
8. https://kartoweb.itc.nl/geometrics/Map projections/body.htm
9. https://en.wikipedia.org/wiki/Haversine_formula
10. https://doc.arcgis.com/en/data-appliance/latest/imagery-elevation/common-attributes.htm